

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2011 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 2 (13 iulie 2011)	2
SUBIECTUL I (30 de puncte)	2
1. Conexitatea grafurilor neorientate	2
2. Programare: Numere prime (tprim)	2
SUBIECTUL al II-lea (30 de puncte)	4
1. Structura sistemelor de calcul: Dispozitive periferice	4
2. Structuri de date: Înregistrare (produs)	4
3. Algoritm eficient: Drum de sumă maximă în matrice pătratică (DP)	4

I. Rezolvare Titularizare Varianta 2 (13 iulie 2011)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Conexitatea grafurilor neorientate

• Definiții:

- **Graf neorientat:** O pereche $G = (V, E)$ de noduri și muchii.
- **Adiacență:** Relația dintre două noduri conectate printr-o muchie.
- **Lanț:** O succesiune de noduri adiacente.
- **Conexitate:** Un graf este conex dacă între oricare două noduri există cel puțin un lanț.
- **Componentă conexă:** Un subgraf conex maximal.

• Aplicații practice:

- **Rețele sociale:** Determinarea grupurilor de prieteni interconectați.
- **Infrastructură:** Verificarea dacă toate orașele sunt legate rutier/feroviar.

- **Algoritm de determinare:** BFS sau DFS pentru identificarea componentelor conexe prin parcurgeri succesive.

—

2. Programare: Numere prime (tprim)

- **Descriere:** Programul determină al N -lea număr prim. Pentru optimizare la determinarea apartenenței lui P , se generează al N -lea prim (limita superioară). Dacă $P \leq t_{\text{prim}}(N)$ și P este prim, atunci P se află în primele N numere prime, altfel nu.

Soluție C++:

```
#include <iostream>
using namespace std;

bool isPrime(long long val) {
    if (val < 2) return false;
    for (long long d = 2; d * d <= val; ++d) {
        if (val % d == 0) return false;
    }
    return true;
}

int tprim(int n) {
    int count = 0;
    int val = 2;
    while (true) {
        if (isPrime(val)) {
            count++;
            if (count == n) return val;
        }
        val++;
    }
}

int main() {
    long long p;
    int n;
    if (!(cin >> p >> n)) return 0;
```

```

    long long limit = tprim(n);
    if (p <= limit && isPrime(p)) {
        cout << "DA\n";
    } else {
        cout << "NU\n";
    }
    return 0;
}

```

Soluție Pascal:

```

program NumerePrime;
var
    p: int64;
    n: integer;
    limit: int64;

function isPrime(val: int64): boolean;
var d: int64;
begin
    if val < 2 then
        begin
            isPrime := false;
            exit;
        end;
    d := 2;
    while d * d <= val do
        begin
            if val mod d = 0 then
                begin
                    isPrime := false;
                    exit;
                end;
            d := d + 1;
        end;
    isPrime := true;
end;

function tprim(n_val: integer): integer;
var
    count, val: integer;
begin
    count := 0;
    val := 2;
    while true do
        begin
            if isPrime(val) then
                begin
                    count := count + 1;
                    if count = n_val then
                        begin
                            tprim := val;
                            exit;
                        end;
                end;
            val := val + 1;
        end;
end;

```

```

    end;
end;

begin
    readln(p, n);
    limit := tprim(n);
    if (p <= limit) and isPrime(p) then
        writeln('DA')
    else
        writeln('NU');
    end.

```

SUBIECTUL al II-lea (30 de puncte)

1. Structura sistemelor de calcul: Dispozitive periferice

- **Rol:** Interfață I/O pentru comunicarea datelor cu UCP.
 - **Clasificare:**
 - **Intrare:** Tastatură, Mouse.
 - **Ieșire:** Monitor, Imprimantă.
 - **Stocare/Mixte:** Placă de rețea.
 - **Parametri:** Rezoluție DPI monitor, polling rate mouse, viteza de printare (PPM).
-

2. Structuri de date: Înregistrare (produs)

- **C++:**

```

struct produs {
    char denumire[100];
    double pret;
};
produs x[2000];
// Afișare preț pe poziția 10
cout << x[9].pret; // 0-indexed sau x[10].pret dacă e 1-indexed

```

- **Pascal:**

```

type
    produs = record
        denumire: string;
        pret: real;
    end;
var
    x: array[1..2000] of produs;
// Afișare preț pe poziția 10
writeln(x[10].pret);

```

3. Algoritm eficient: Drum de sumă maximă în matrice pătratică (DP)

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>

```

```
using namespace std;

int main() {
    ifstream fin("NR.TXT");
    int n;
    if (!(fin >> n)) return 0;
    vector<vector<long long>> A(n, vector<long long>(n));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            fin >> A[i][j];
        }
    }
    fin.close();

    vector<vector<long long>> dp(n, vector<long long>(n, 0));
    for (int j = 0; j < n; ++j) {
        dp[0][j] = A[0][j];
    }

    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            long long prev_max = dp[i-1][j];
            if (j > 0) prev_max = max(prev_max, dp[i-1][j-1]);
            if (j < n - 1) prev_max = max(prev_max, dp[i-1][j+1]);
            dp[i][j] = A[i][j] + prev_max;
        }
    }

    long long ans = dp[n-1][0];
    for (int j = 1; j < n; ++j) {
        ans = max(ans, dp[n-1][j]);
    }

    cout << ans << endl;
    return 0;
}
```

Soluție Pascal:

```
program MaxPathSum;
var
    fin: text;
    n, i, j: integer;
    A: array[1..100, 1..100] of integer;
    dp: array[1..100, 1..100] of int64;
    prev_max, ans: int64;
begin
    assign(fin, 'NR.TXT'); reset(fin);
    read(fin, n);
    for i := 1 to n do
        for j := 1 to n do
            read(fin, A[i, j]);
    close(fin);

    for j := 1 to n do dp[1, j] := A[1, j];

    for i := 2 to n do
```

```
begin
  for j := 1 to n do
    begin
      prev_max := dp[i - 1, j];
      if j > 1 then
        begin
          if dp[i - 1, j - 1] > prev_max then prev_max := dp[i - 1, j - 1];
        end;
      if j < n then
        begin
          if dp[i - 1, j + 1] > prev_max then prev_max := dp[i - 1, j + 1];
        end;
      dp[i, j] := A[i, j] + prev_max;
    end;
  end;

  ans := dp[n, 1];
  for j := 2 to n do
    begin
      if dp[n, j] > ans then ans := dp[n, j];
    end;
  writeln(ans);
end.
```